SentinelScan — Web Security Assessment & Vulnerability Analysis Platform

> ***SentinelScan*** *is an enterprise-grade, defensive web security assessment platform. It enables security teams, DevSecOps engineers, and web administrators to conduct rigorous, non-destructive vulnerability audits against web applications with zero credential leakage, strict SSRF immunity, and deterministic risk scoring.*

■ **GitHub Repository**: **https://github.com/koyel2004/webscan** (https://github.com/koyel2004/webscan)

■ Table of Contents

1. **Executive Summary & Problem Statement** (#1-executive-summary--problem-statement)
2. **Core Capabilities & Defensive Principles** (#2-core-capabilities--defensive-principles)
3. **The 7 Modular Security Check Suites** (#3-the-7-modular-security-check-suites)
4. **SSRF Protection & Defensive Guarantees** (#4-ssrf-protection--defensive-guarantees)
5. **Deterministic Scoring & Grading Algorithm** (#5-deterministic-scoring--grading-algorithm)
6. **System Architecture** (#6-system-architecture)
7. **Technology Stack** (#7-technology-stack)
8. **Quick Start & Local Setup** (#8-quick-start--local-setup)
9. **Running with Docker & Docker Compose** (#9-running-with-docker--docker-compose)
10. **Automated Test Suite & Quality Assurance** (#10-automated-test-suite--quality-assurance)
11. **REST API Reference** (#11-rest-api-reference)
12. **Frontend Workspaces & User Interface** (#12-frontend-workspaces--user-interface)
13. **Dual Reporting Engine (PDF & JSON)** (#13-dual-reporting-engine-pdf--json)
14. **Documentation Index** (#14-documentation-index)
15. **Responsible Disclosure & Ethical Policy** (#15-responsible-disclosure--ethical-policy)

1. Executive Summary & Problem Statement

The Problem

Traditional vulnerability scanners often suffer from several critical shortcomings:

- **Overly Aggressive / Destructive**: Many scanners blindly inject malformed payloads, attempt active exploits, or flood endpoints, risking data corruption, service downtime, or account lockouts.
- **Credential & Token Leakage**: Active scanning tools frequently log full raw HTTP headers, cookies, and bearer tokens in plain text, turning scan databases into high-value breach targets.
- **SSRF Vulnerabilities**: In web-based security platforms, scanning backends frequently fall victim to Server-Side Request Forgery (SSRF) when attackers feed internal network addresses (`127.0.0.1`, `169.254.169.254`, `10.0.0.0/8`), weaponizing the scanner to breach internal cloud infrastructure.
- **Opaque Risk Scores**: Arbitrary heuristics without clear mapping to industry taxonomies leave developers confused about remediation priorities.

The SentinelScan Solution

SentinelScan was engineered from the ground up to be a **purely defensive, non-destructive, and secure security auditing platform**:

- **100% Non-Destructive**: Audits web application posture using passive inspection, protocol handshakes, and RFC-compliant queries. Zero exploits, zero SQL injections, zero fuzzing.
- **Pre-Resolution SSRF Barrier**: Resolves target hostnames against an internal subnet blacklist *before* dispatching any socket connection. Blocks loopback, private networks, cloud metadata, and dangerous ports.

• **Automated Data Redaction**: Automatically redacts sensitive session cookies, bearer tokens, and passwords (`[REDACTED]`) across all logs, databases, and generated reports.

• **Standardized Taxonomies**: Every finding is explicitly mapped to **OWASP Top 10 (2021)** and **MITRE Common Weakness Enumerations (CWE)** with remediation code snippets.

• **Dual Reporting Engine**: Instant generation of executive-ready PDF reports with visual risk charts, plus machine-readable JSON exports for DevSecOps CI/CD pipelines.

---

2. Core Capabilities & Defensive Principles

SentinelScan adheres strictly to defensive cybersecurity best practices:

• ■■ **Non-Destructive Scanning**: Never executes active exploits, remote code execution, SQL injections, or cross-site scripting attacks.

• ■ **Zero Credential Attacks**: Never performs brute-force password spraying or dictionary attacks.

• ■■ **Resource Bound**: Enforces a strict 2 MB response size cap and 30-second timeout per probe to prevent Denial of Service (DoS) conditions on target systems.

• ■■ **Consent-Driven Scanning**: Enforces explicit operator confirmation acknowledging authorization to audit the designated target domain.

---

3. The 7 Modular Security Check Suites

SentinelScan executes 7 focused security suites in parallel under an asynchronous semaphore-controlled engine:

Check Suite	Key Checks & Probes	CWE Mapping	OWASP Top 10 (2021)	Default Severity
Check A: HTTPS Redirect	Validates HTTP -> HTTPS redirection, checks for permanent status codes (`301`/`308` vs temporary `302`), verifies immediate TLS upgrade without intermediate unencrypted hops.	CWE-311, CWE-319	A02: Cryptographic Failures	Medium / Low
Check B: Security Headers	Audits presence, syntax, and strength of essential defensive headers: `Content-Security-Policy` (CSP), `Strict-Transport-Security` (HSTS), `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, and `Permissions-Policy`.	CWE-693, CWE-1021	A05: Security Misconfiguration	High / Medium
Check C: Cookie Security	Inspects all `Set-Cookie` directives for critical defensive flags: `Secure`, `HttpOnly`, and `SameSite` (`Strict`/`Lax`). Flags cookies exposed over plain HTTP or accessible to client-side scripts. All cookie values are permanently redacted.	CWE-614, CWE-1004, CWE-1275	A05: Security Misconfiguration	High / Medium
Check D: CORS Policy	Probes CORS behavior with arbitrary origin reflection tests, `Origin: null`, and credential reflection (`Access-Control-Allow-Credentials: true` with wildcard or untrusted origins).	CWE-346, CWE-942	A01: Broken Access Control	High / Medium
Check E: TLS Inspection	Audits cryptographic parameters: certificate expiry countdown, Subject Alternative Name (SAN) coverage, root trust chain validation, and deprecated protocol detection.	CWE-295, CWE-298	A02: Cryptographic Failures	Critical / High
Check F: Information Disclosure	Scans response headers and body markers for version disclosure: detailed `Server` banners (e.g., `Apache/2.4.41`), `X-Powered-By`, `X-AspNet-Version`, and debug fingerprints that aid adversary reconnaissance.	CWE-200	A05: Security Misconfiguration	Low / Info
Check G: Response Hygiene	Detects unhandled application errors, detailed framework stack traces (Django, Flask, Spring, Express, ASP.NET), and unintended protocol fallback behaviors.	CWE-209	A05: Security Misconfiguration	Medium / Low

---

4. SSRF Protection & Defensive Guarantees

SentinelScan features an immutable multi-tier target validation gate (`TargetValidator` (backend/app/services/scanner/target_validator.py)):

```
Target URL ■■■ 1. Syntax Validation ■■■ 2. Port Filtering ■■■ 3. DNS Pre-Resolution ■■■ 4. Safe HTTPX Dispatch
(Scheme: http/https) (Block 22,25,3389, (Block 127.0.0.1, (2MB Limit, 30s Timeout,
(Reject credentials) 6379, 27017...) 10.0.0.0/8, 169.254...) Manual Redirect Audit)
```

1. **Syntax Check**: Accepts only valid `http` or `https` schemes; strictly rejects alternative URI schemes (`file://`, `ftp://`, `gopher://`, `ldap://`), raw hex IP notation, or embedded user credentials (`http://user:pass@host`).
2. **Port Restrictions**: Blocks scanning of internal management ports (SSH 22, SMTP 25, Telnet 23, Redis 6379, MongoDB 27017, MySQL 3306, Postgres 5432, RDP 3389).
3. **DNS IP Pre-Resolution**: Performs synchronous DNS lookup via `socket.getaddrinfo()` and evaluates every resolved IP address against RFC blacklists:
- Loopback: `127.0.0.0/8`, `::1`
- Private RFC 1918: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`
- Link-Local & Cloud Metadata: `169.254.0.0/16`, `fe80::/10`
- Carrier-Grade NAT: `100.64.0.0/10`
- Documentation & Broadcast: `192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`, `240.0.0.0/4`, `255.255.255.255`
4. **Manual Redirect Inspection**: Disables automatic redirect following in HTTP clients. Every redirection target URL is independently inspected and resolved against the SSRF filter before a subsequent probe is dispatched.
5. **Sensitive Data Redaction Filter**: A custom `logging.Filter` (`SensitiveDataFilter`) automatically scrubs bearer tokens, basic auth credentials, and cookie values from all standard and error output streams.

---

5. Deterministic Scoring & Grading Algorithm

SentinelScan calculates security grades using a transparent, deterministic algorithm:

\text{Final Score} = \max\left(0, 100 - \sum \text{Severity Deductions}\right)

Severity Weights:

- **Critical**: -25 points
- **High**: -15 points
- **Medium**: -8 points
- **Low**: -3 points
- **Info**: 0 points

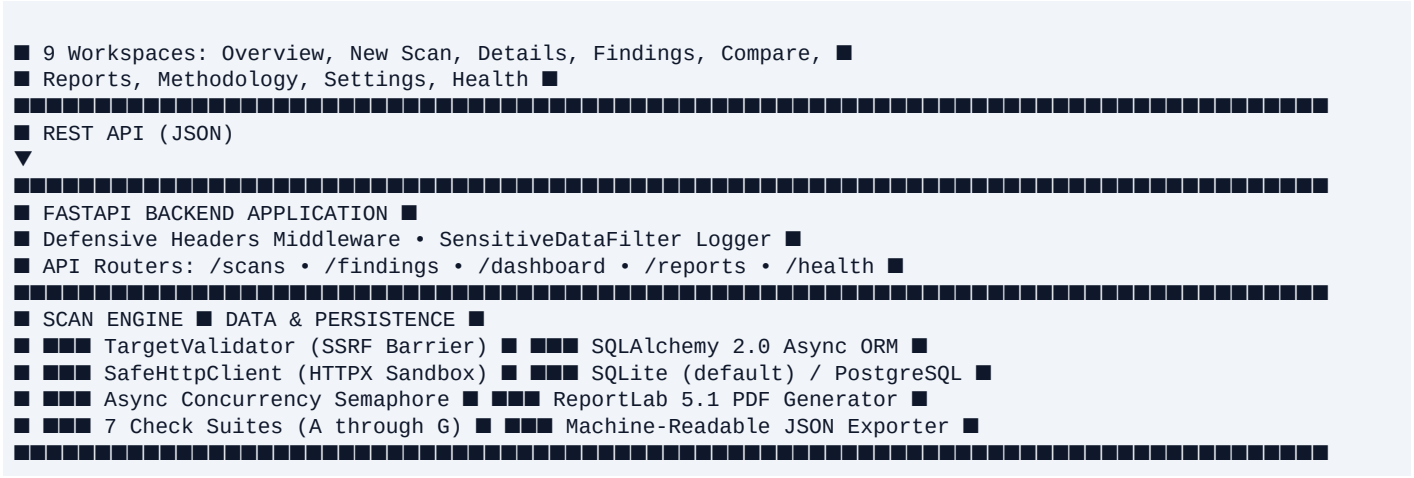
Letter Grade Scale:

Letter Grade	Score Range	Posture Assessment
A+	95 - 100	Exemplary defense-in-depth posture; all critical headers, TLS, and cookies enforced.
A	90 - 94	Excellent posture; minor informational or non-critical headers omitted.
B	80 - 89	Good posture; missing secondary headers (e.g., Referrer-Policy or Permissions-Policy).
C	70 - 79	Moderate risk; missing critical headers (CSP or HSTS) or cookie flags omitted.
D	60 - 69	Elevated risk; multiple high-severity misconfigurations or weak TLS parameters.
F	0 - 59	Severe risk; critical vulnerabilities present (expired TLS, CORS wildcard with credentials, plain HTTP).

---

6. System Architecture

```
■■■
```



7. Technology Stack

Backend

- **Language**: Python 3.11 / 3.12
- **Web Framework**: FastAPI 0.110+ (Asynchronous ASGI)
- **ASGI Server**: Uvicorn
- **Database ORM**: SQLAlchemy 2.0 with Alembic schema migrations
- **Data Validation**: Pydantic v2
- **HTTP Client**: HTTPX (Asynchronous, configured with defensive timeouts and connection pooling)
- **Report Generation**: ReportLab 5.1 (High-resolution, vector-based executive PDF reports)
- **Code Quality**: Ruff (Linting & formatting), Pytest (Test runner)

Frontend

- **Core**: React 19.0.0, TypeScript 5.8
- **Build System**: Vite 6.2
- **Styling**: Tailwind CSS v4, Vanilla CSS design tokens
- **Icons**: Lucide React
- **Data Visualization**: Recharts (Pie charts, bar charts, area charts, gauge indicators)
- **HTTP Client**: Axios with centralized error handling

DevOps & Infrastructure

- **Containerization**: Multi-stage Dockerfiles with unprivileged runtime users (`sentinel` user, non-root)
- **Orchestration**: Docker Compose for multi-container frontend + backend deployment
- **Reverse Proxy**: NGINX Alpine for frontend asset delivery and API proxying

8. Quick Start & Local Setup

Prerequisites

- Python 3.11 or higher
- Node.js 20 or higher & npm

1. Clone the Repository

```
git clone https://github.com/koyel2004/webscan.git
cd webscan
```

2. Backend Setup

```
# Navigate to backend directory
cd backend

# Create and activate Python virtual environment
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Run database migrations
alembic upgrade head

# Start the FastAPI backend server
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

- Backend API: `http://localhost:8000`
- Swagger Interactive Docs: `http://localhost:8000/docs`
- ReDoc API Docs: `http://localhost:8000/redoc`

3. Frontend Setup

```
# In a new terminal, navigate to frontend directory
cd frontend

# Install frontend dependencies
npm install --legacy-peer-deps

# Start the Vite development server
npm run dev
```

- Open your browser and navigate to: `http://localhost:5173`

---

9. Running with Docker & Docker Compose

SentinelScan is fully containerized for seamless, one-command deployment:

```
# Build and start all services
docker compose up -d --build

# Verify running containers
docker compose ps
```

- **Frontend Dashboard**: `http://localhost` (Port 80)
- **Backend API & Docs**: `http://localhost:8000/docs`

To view application logs or shutdown:

```
# View backend logs in real time
docker compose logs -f backend

# Stop and remove containers
docker compose down
```

---

10. Automated Test Suite & Quality Assurance

SentinelScan features a comprehensive automated test suite achieving **100% pass rate** across all core security and engine modules:

```
cd backend
source venv/bin/activate

# Run full pytest test suite
pytest -v
```

Test Coverage Breakdown (23/23 Passing):

- `tests/test_validator.py` (6 tests): Validates SSRF defense, loopback blocking, private IP rejection, scheme validation, port restrictions.
- `tests/test_checks.py` (7 tests): Validates all 7 check suites (HTTPS redirect, security headers, cookie flags, CORS policy, TLS verification, information disclosure, response hygiene).
- `tests/test_scoring.py` (3 tests): Validates score calculations, severity point deduction, and letter grade thresholds.
- `tests/test_redaction.py` (2 tests): Validates cookie value redaction and sensitive log masking.
- `tests/test_api.py` (3 tests): Validates scan initiation, scan cancellation, findings retrieval, and dashboard metrics.
- `tests/test_reports.py` (2 tests): Validates PDF report compilation and JSON export generation.

Code Style & Linting:

```
# Run Ruff linting
ruff check app tests
```

- **Result**: `0 errors, all checks passed.`

---

11. REST API Reference

Method	Endpoint	Description	Request Body / Parameters
`GET`	`/api/v1/health`	System health check and database connectivity	None
`GET`	`/api/v1/methodology`	List of 7 check suites with CWE/OWASP mappings	None
`GET`	`/api/v1/dashboard`	Aggregated scan stats, grade distribution, recent scans	None
`POST`	`/api/v1/dashboard/seed-demo`	Seed realistic mock scans for testing & demo	None
`POST`	`/api/v1/scans`	Dispatch a new defensive security scan	`{"target_url": "...", "authorized": true}`
`GET`	`/api/v1/scans`	List scan history with pagination and status filters	`?skip=0&limit=20&status=completed`
`GET`	`/api/v1/scans/{id}`	Detailed scan record, execution timeline, and findings	Scan ID (path)
`POST`	`/api/v1/scans/{id}/cancel`	Cancel an ongoing scan	Scan ID (path)
`DELETE`	`/api/v1/scans/{id}`	Delete scan record and cascade-delete findings	Scan ID (path)
`GET`	`/api/v1/findings`	Search and filter findings by severity or category	`?severity=high&category=headers`
`GET`	`/api/v1/reports/{id}/pdf`	Download formatted executive and technical PDF report	Scan ID (path)
`GET`	`/api/v1/reports/{id}/json`	Download structured machine-readable JSON report	Scan ID (path)

---

12. Frontend Workspaces & User Interface

SentinelScan provides 9 purpose-built navigation workspaces:

1. **Dashboard & Overview**: High-level security posture summary, average risk score, grade breakdown, and recent activity feed.
2. **Launch Security Scan**: Clean interface for entering target URLs, selecting check suites, and confirming scan authorization.
3. **Active & Historical Scans**: Searchable, filterable audit log of all completed, running, and failed scans.

4. **Scan Deep Dive**: Granular execution breakdown showing check-by-check pass/fail status, duration, raw evidence, and score impact.
5. **Vulnerability Explorer**: Searchable inventory of all detected vulnerabilities across all targets, filterable by severity, CWE, or category.
6. **Scan Comparison**: Side-by-side posture comparison between different scans or before/after remediation runs.
7. **Reporting Center**: One-click download of executive PDF reports and machine-readable JSON exports.
8. **Methodology & Taxonomies**: Educational interactive reference detailing the 7 check suites, RFC citations, and remediation blueprints.
9. **Settings & Health**: System diagnostics, worker concurrency settings, and active database connection metrics.

---

13. Dual Reporting Engine (PDF & JSON)

Executive & Technical PDF Report

Generated dynamically using ReportLab 5.1 with:

- Executive cover page with target metadata, scan duration, and letter grade badge.
- Visual risk score dial and severity breakdown summary table.
- Detailed findings table with title, severity, category, CWE ID, and OWASP mapping.
- Granular remediation instructions with secure code snippets for each finding.
- Disclaimer and methodology audit log.

Machine-Readable JSON Export

Export compliant with DevSecOps ingestion standards:


```
{
"scan_id": "9c8a7f4e-2b1d-4a3c-8e9f-1a2b3c4d5e6f",
"target_url": "https://example.com",
"status": "completed",
"grade": "B",
"score": 82,
"summary": {
"total_findings": 3,
"critical": 0,
"high": 1,
"medium": 2,
"low": 0
},
"findings": [
{
"check_id": "check_b_headers",
"severity": "high",
"title": "Missing Content-Security-Policy Header",
"cwe": "CWE-693",
"owasp": "A05:2021 Security Misconfiguration",
"remediation": "Configure Content-Security-Policy with appropriate directives."
}
]
}
```


---

14. Documentation Index

The `docs/` folder contains comprehensive documentation for developers, security engineers, and compliance auditors:

- ■ **System Architecture & Design Document** (docs/architecture.md) — Comprehensive architectural specification and component boundaries.
- ■■ **Security Assessment Methodology** (docs/security-methodology.md) — Technical methodology for each of the 7 security check suites.

- ■ **REST API Specification** (docs/api-reference.md) — Complete endpoint schemas, request bodies, and response models.
- ■ **Threat Model & Defensive Guarantees** (docs/threat-model.md) — Threat matrix, SSRF defenses, and data protection policies.
- ■ **User Guide & Operational Manual** (docs/user-guide.md) — Step-by-step walkthrough for security operators.

---

15. Responsible Disclosure & Ethical Policy

> ■■ ***ETHICAL USE NOTICE***:
>
> *SentinelScan is designed solely for defensive cybersecurity assessment, risk evaluation, and DevSecOps compliance auditing. Users must only evaluate target applications that they own, operate, or have received explicit written authorization to assess.*
>
> >
>
> *Probing third-party infrastructure without authorization is unlawful in many jurisdictions and strictly prohibited by this software's operational principles. The developers assume no liability for misuse.*

■ Author & Repository Information

- **Developer**: Ankita Kala (@**koyel2004** (https://github.com/koyel2004))
- **GitHub Repository**: **https://github.com/koyel2004/webscan** (https://github.com/koyel2004/webscan)
- **License**: MIT License